`

EXPERIMENT NO.4

Name:Bhgyesh Patil
Class:D20A
Roll No:3
Batch:A

Aim:Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:-

1. Primitive Data Types, Variables, Functions - pure, view

Primitive (Value) Data Types in Solidity include:

- bool → true/false
- int / uint (signed/unsigned integers, e.g., uint256 is most common)
- address → 20-byte Ethereum address
- bytes1 to bytes32 → fixed-size byte arrays
- string → dynamic UTF-8 encoded text (reference type, but often grouped here)

Variables are declared with a type and can be:

- **State variables** → stored permanently on the blockchain (expensive)
- **Local variables** → exist only during function execution

Functions can be marked as:

- pure → does not read or write state (computes only from inputs; cheapest gas)
  - view → reads state but does not modify it (e.g., getters; no gas when called externally)

`

Example

```
function getResult() public view returns (uint product, uint sum) {
    product = num1 * num2;  // reads state
    sum = num1 + num2;
}

function pureCalc(uint a, uint b) public pure returns (uint) {
    return a + b;  // no state access
}
```

2. Inputs and Outputs to Functions

Functions in Solidity can take **parameters** (inputs) and return **values** (outputs).

- **Inputs**: Declared in parentheses; can use data locations like memory or calldata for reference types.
- **Outputs**: Declared after returns keyword; can return multiple values.

Example:

```
function add(uint a, uint b) public pure returns (uint sum) {
    sum = a + b;
}
// Multiple outputs
function getValues() public view returns (uint, string memory) {
    return (age, name);
}
```

`

- Use calldata for external calls (cheaper, read-only).
- Use memory for temporary copies inside functions.

3. Visibility, Modifiers and Constructors

Visibility Specifiers (for functions and state variables):

- **public** → anyone can call/read (default for state vars creates getter)
- **private** → only inside current contract
- **internal** → current + derived (child) contracts
- **external** → only external calls (cheaper for large data)

Modifiers → reusable code blocks that run before/after function body (e.g., access control).

Use _; to insert function body.

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
}

function restricted() public onlyOwner { ... }
```

Constructors → special function that runs once on deployment (initializes state).

- Syntax: constructor() { ... } (older: same name as contract)

`

4. Control Flow: if-else, loops

Solidity supports standard control structures:

if-else:

```
if (condition) {
   // true
} else if (another) {
   // else-if
} else {
   // false
}
```

Loops:

- for (uint i = 0; i < 10; i++) { ... }
- while (condition) { ... }
- do { ... } while (condition);

Avoid unbounded loops (gas limit risk). Use break / continue when needed.

5. Data Structures: Arrays, Mappings, structs, enums

- **Arrays**
  - Fixed: uint[5] arr;
  - Dynamic: uint[] arr; (use .push(), .pop(), .length)
- **Mappings** → key-value store (like hash table)
  mapping(address => uint) public balances;

  Keys can be most types; no iteration possible.

`

Structs → custom composite types

```
struct User {
    address wallet;
    uint balance;
    bool active;
}
User public owner;
```

Enums → named constants (integers under the hood)

```
enum Status { Pending, Active, Cancelled }
Status public state = Status.Pending;
```

6. Data Locations

Solidity has three main **data locations** for reference types (arrays, structs, mappings, strings):

- **storage** → permanent blockchain storage (persistent, expensive)
  Default for state variables.
  - **memory** → temporary, function lifetime only (deleted after execution)
    Cheap; used for local variables & function args/returns.
- **calldata** → read-only, non-modifiable area for function call data
  Cheapest for external function parameters (immutable copy of tx data).

Rule:

- State vars → always storage
- Function args → prefer calldata (external) or memory
- Local reference vars → must specify location

`

7. Transactions: Ether and wei, Gas and Gas Price, Sending Transactions

- **Ether units** → smallest is **wei** (1 ETH = 10^{18} wei) Other: gwei (10^{9} wei), commonly used for gas prices.
  - **Gas** → computational effort unit
    - **Gas Limit** → max gas willing to spend (set by sender)
    - **Gas Price** → price per gas unit (in wei/gwei; set by sender)
    - Total fee = gas used × gas price
  - **msg.value** → amount of wei sent with transaction
- **Sending Ether** → use .transfer(), .send(), or .call{value: amount}("") (recommended low-level call)

Example:

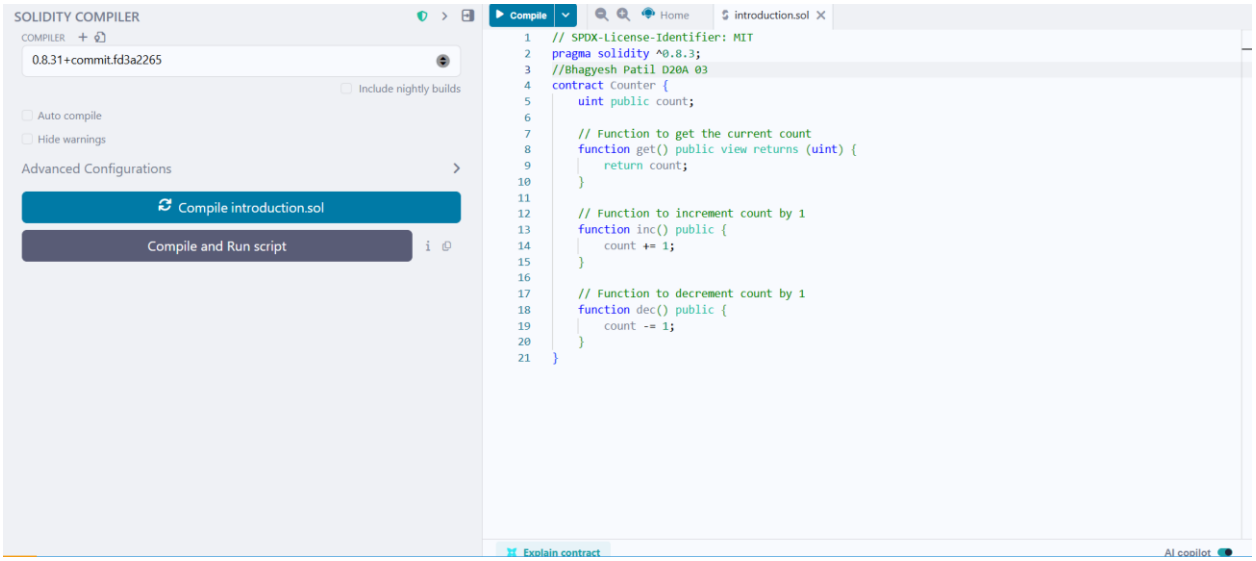
```
function sendEther(address payable recipient) public payable {
   recipient.transfer(msg.value);
}
```

- Use payable for addresses/functions that receive Ether.
- tx.gasprice → current gas price (global var)

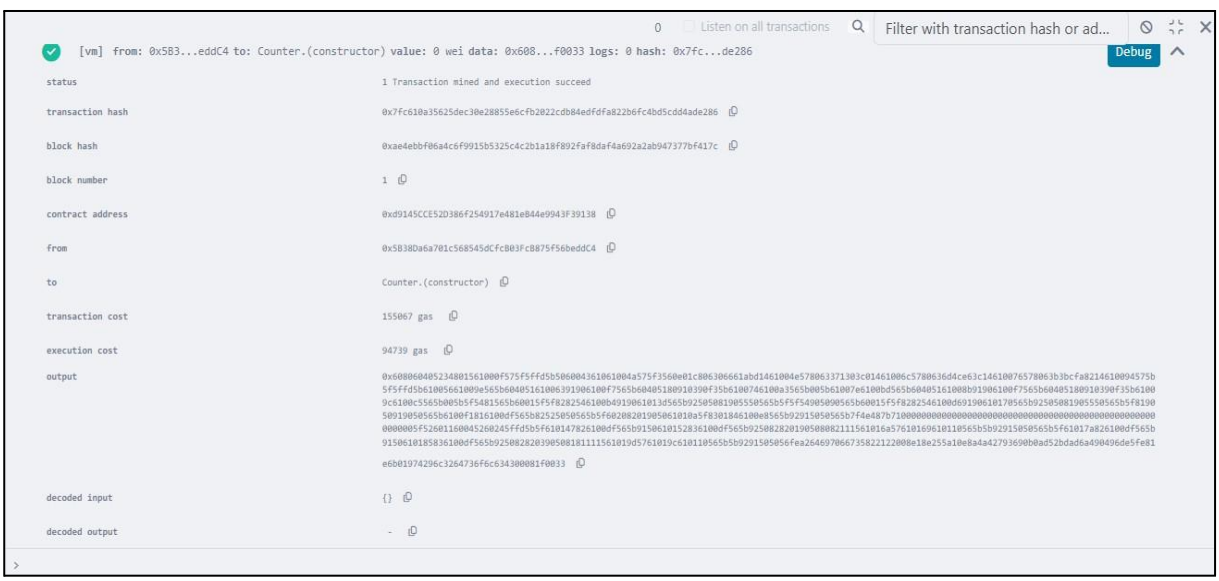
`

Implementation:-

- Tutorial no. 1 – Compile the code



- Tutorial No.1-Deploy the contract

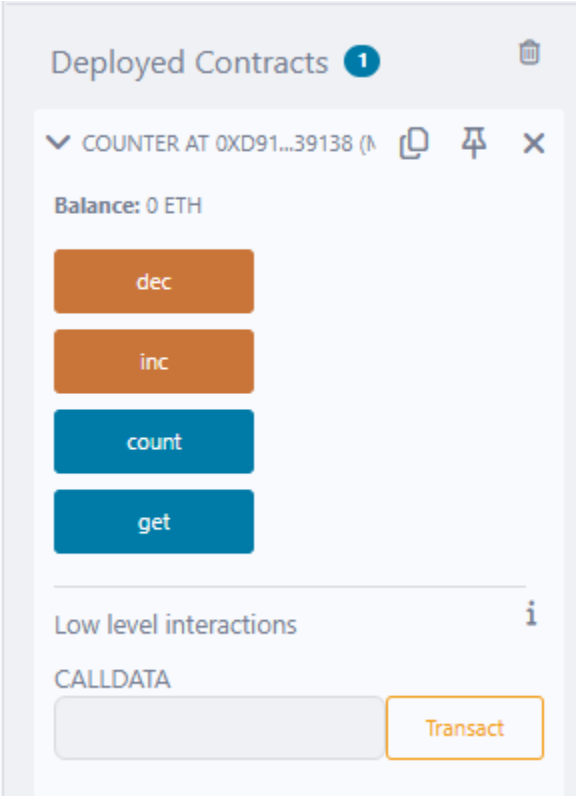


`

- Tutorial no. 1 – get

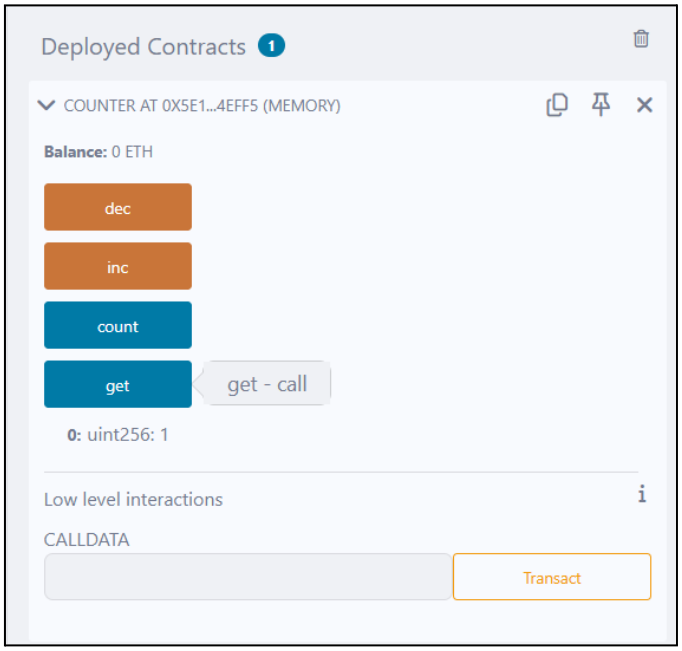
```
CALL   [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c

from                    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                      Counter.get() 0xd9145CCE52D386f254917e481eB44e9943F39138
execution cost          2453 gas (Cost only applies when called by a contract)
input                   0x6d4...ce63c
output                  0x0000000000000000000000000000000000000000000000000000000000000000
decoded input           {}
decoded output          {
                            "0": "uint256: 0"
                        }
logs                    []
raw logs                []
```

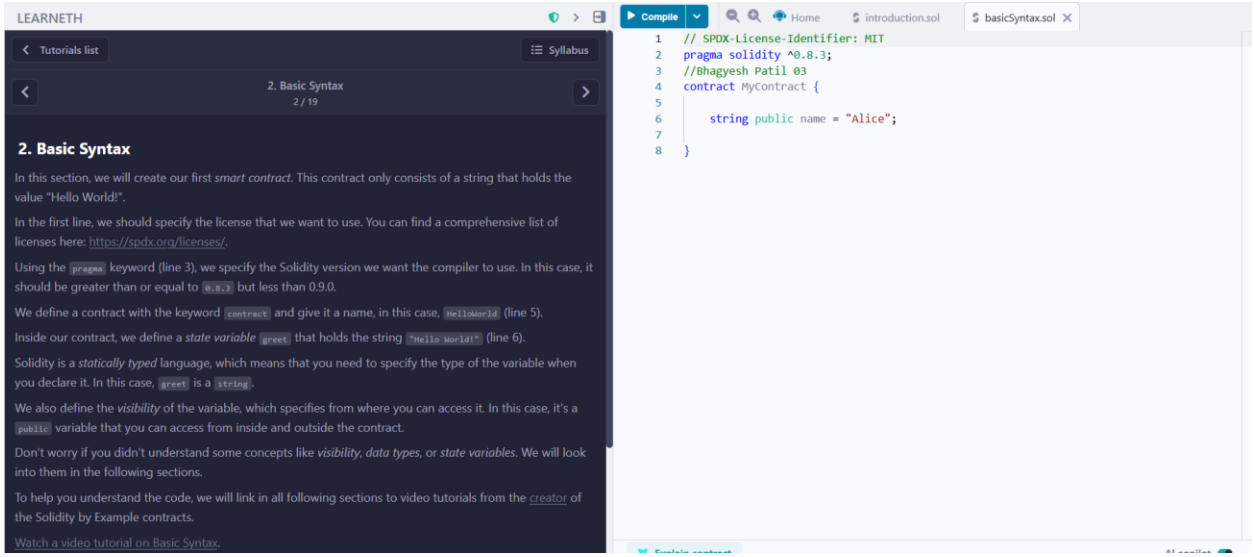


`

- Tutorial no. 1 – Decrement



- Tutorial no.2



`

- Tutorial no.3

LEARNETH

Tutorials list | Syllabus

3. Primitive Data Types
3 / 19

3. Primitive Data Types

In this section, we will show you Solidity's primitive data types, how to declare them, and their characteristics.

bool

You can declare data a boolean type by using the keyword 'bool'. Booleans can either have the value `true` or `false`.

uint

We use the keywords `uint` and `uint8` to `uint256` to declare an *unsigned integer type* (they don't have a sign, unlike -12, for example). Uints are integers that are positive or zero and range from 8 bits to 256 bits. The type `uint` is the same as `uint256`.

int

We use the keywords `int` and `int8` to `int256` to declare an integer type. Integers can be positive, negative, or zero and range from 8 bits to 256 bits. The type `int` is the same as `int256`.

address

Variables of the type `address` hold a 20-byte value, which is the size of an Ethereum address. There is also a special kind of Ethereum address, `address payable`, which can receive ether from the contract.

All these data types have default values, as shown in the contract (line 29).

You can learn more about these data types as well as *Fixed Point Numbers*, *Byte Arrays*, *Strings*, and more in

Compile | Home | introduction.sol | basicSyntax.sol | primitiveDataTypes.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3  // Bhagyesh Patil 03
4  contract Primitives {
5
6      bool public boo = true;
7
8      uint public u = 123;
9
10     int public i = -123;
11
12     address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
13
14     // ✅ Assignment variables
15
16     address public newAddr = 0x0000000000000000000000000000000000000001;
17
18     int public neg = -5;
19
20     uint8 public newU = 0;
21
22 }
```

- Tutorial no.4

LEARNETH

Tutorials list | Syllabus

4. Variables
4 / 19

4. Variables

There are three different types of variables in Solidity: *State Variables*, *Local Variables*, and *Global Variables*.

1. State Variables

State Variables are stored in the contract *storage* and thereby on the blockchain. They are declared inside the contract but outside the function. This contract has two state variables, the string `text` (line 6) and the uint `num` (line 7).

2. Local Variables

Local Variables are stored in the *memory* and their values are only accessible within the function they are defined in. Local Variables are not stored on the blockchain. In this contract, the uint `i` (line 11) is a local variable.

3. Global Variables

Global Variables, also called *Special Variables*, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the Solidity documentation.

Compile | introduction.sol | basicSyntax.sol | primitiveDataTypes.sol | variables.sol 3

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3  //Bhagyesh Patil 03
4  contract Variables {
5
6      string public text = "Hello";
7      uint public num = 123;
8
9      uint public blockNumber;
10     function doSomething() public {    22334 gas
11
12         uint i = 456;
13
14         uint timestamp = block.timestamp;
15
16         address sender = msg.sender;
17
18         blockNumber = block.number;
19     }
20 }
```

`

- Tutorial no.5

LEARNETH

Tutorials list | Syllabus

5.1 Functions - Reading and Writing to a State Variable
5 / 19

`pure`. For example, in this contract, the `get` function (line 14) marked `view` that only returns `num` does not change the state.

To define a function, use the `function` keyword followed by a unique name.

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on Functions.

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer | Show answer

Next

Well done! No errors.

Compiled | asicSyntax.sol | primitiveDataTypes.sol | variables.sol 3 | readAndWrite.sol

```
1  // Bhagyesh Patil 03
2  // SPDX-License-Identifier: MIT
3  pragma solidity ^0.8.3;
4
5  contract SimpleStorage {
6
7      uint public num;
8
9      bool public b = true;
10
11     function set(uint _num) public {    22536 gas
12         num = _num;
13     }
14
15     function get() public view returns (uint) {    2475 gas
16         return num;
17     }
18
19     function get_b() public view returns (bool) {    2539 gas
20         return b;
21     }
22 }
```

Explain contract | AI copilot

- Tutorial no.6

LEARNETH

Tutorials list | Syllabus

5.2 Functions - View and Pure
6 / 19

3. Accessing any of the members of block, tx, msg (with the exception of `msg.sig` and `msg.data`).
4. Calling any function not marked pure.
5. Using inline assembly that contains certain opcodes."

From the Solidity documentation.

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | ypes.sol | variables.sol 3 | readAndWrite.sol | viewAndPure.sol

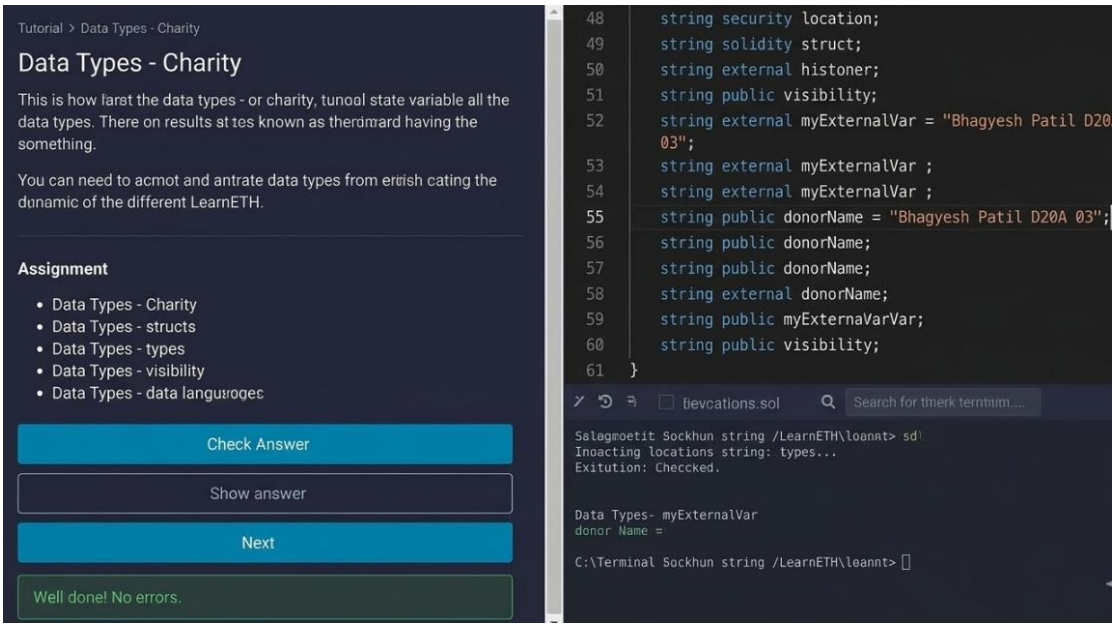
```
1  // Bhagyesh Patil https://remix.ethereum.org/plugins/learneth/index.html#/detail?id=183c
2  // SPDX-License-Identifier: MIT
3  pragma solidity ^0.8.3;
4
5  contract ViewAndPure {
6
7      uint public x = 1;
8
9      function addToX(uint y) public view returns (uint) {    infinite gas
10         return x + y;
11     }
12
13     function add(uint i, uint j) public pure returns (uint) {    infinite gas
14         return i + j;
15     }
16
17     function addToX2(uint y) public {    infinite gas
18         x = x + y;
19     }
20 }
```

Explain contract | AI copilot

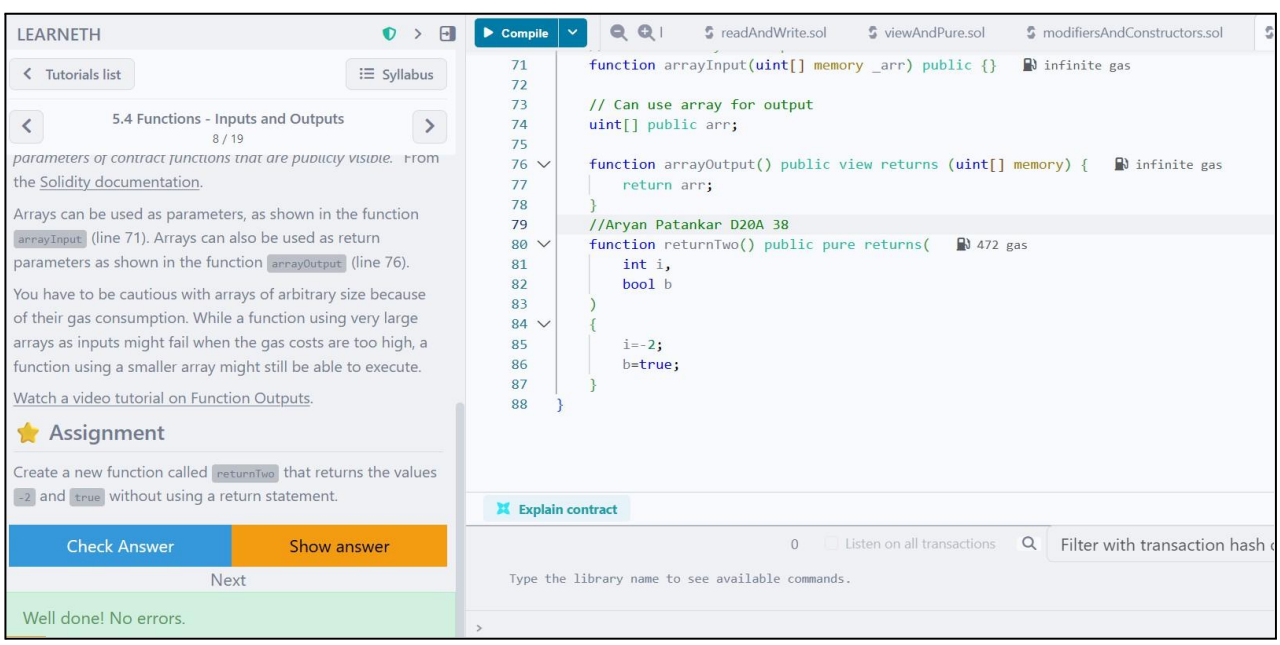
Alert | Initialize as git repo | Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. | RemixAI Copilot (enabled)

`

- Tutorial no.7



- Tutorial no.8



`

- Tutorial no.9

LEARNETH

Tutorials list | Syllabus

6. Visibility
9 / 19

and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer | Show answer

Next

Well done! No errors.

Compile | rite.sol | viewAndPure.sol | modifiersAndConstructors.sol | inputsAndOutputs.sol | visibility

```
52      // string external externalVar = "my external variable";
53  }
54
55  contract Child is Base {
56      // Inherited contracts do not have access to private functions
57      // and state variables.
58      // function testPrivateFunc() public pure returns (string memory) {
59      //     return privateFunc();
60      // }
61
62      // Internal function call be called inside child contracts.
63      //Aryan Patankar D20A 38
64      function testInternalFunc() public pure override returns (string memory) {    infinite gas
65          return internalFunc();
66      }
67
68      function testInternalVar() public view returns(string memory x,string memory y){    infinite gas
69          return(internalVar,publicVar);
70      }
71  }
```

Explain contract | AI copilo

0 | Listen on all transactions | Filter with transaction hash or ad...

Type the library name to see available commands.

- Tutorial no.10

LEARNETH

Tutorials list | Syllabus

7.1 Control Flow - If/Else
10 / 19

the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternery operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer | Show answer

Next

Well done! No errors.

Compile | ewAndPure.sol | modifiersAndConstructors.sol | inputsAndOutputs.sol | visibility.sol

```
10          } else {
11              return 2;
12          }
13      }
14
15      function ternary(uint _x) public pure returns (uint) {    infinite gas
16          // if (_x < 10) {
17          //     return 1;
18          // }
19          // return 2;
20
21          // shorthand way to write if / else statement
22          return _x < 10 ? 1 : 2;
23      }
24      //Aryan Patankar D20A 38
25      function evenCheck(uint _number) public pure returns (bool) {    infinite gas
26          return (_number % 2 == 0) ? true : false;
27      }
28  }
```

Explain contract

0 | Listen on all transactions | Filter with transaction hash or ad

Type the library name to see available commands.

`

- Tutorial no.11

LEARNETH

Tutorials list | Syllabus

7.2 Control Flow - Loops
11 / 19

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer | Show answer

Next

Well done! No errors.

Compile | modifiersAndConstructors.sol | inputsAndOutputs.sol

```
13                break;
14            }
15            count++;
16        }
17
18        // while loop
19        //Bhagyesh Patil D20A 03
20        uint j;
21        while (j < 10) {
22            j++;
23            if (count < 9) {
24                count++;
25            }
26        }
27    }
28
29
30
31
```

Explain contract

0 | Listen on all transactions | Filt

Type the library name to see available commands.

- Tutorial no.12

LEARNETH

Tutorisls list | Syllabus

8.1 Data Structures - Arrays
12 / 19

the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized arrey called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | Constructers.sol | $ inputsAndOutputs.sol | $ visibilityaol | $ ifElse

```
4   contract Array {
5       // Sevoral woys to initialize an array
6       //Bhagyesh Patil D20A 03
7       uint[] public arr;
8       uint[] public arr2 = [1, 2, 3];
9       uint[3] pobiic arr3 = [0, 1, 2];
10
11      // Fixed sized array, all elements Initialize to 0
12      uint[13] public myFixedSizeArr;
13
14      function get(uint i) public view returns (uint) {    infinite gas
15          return arr[i];
16      }
17
18
19      // Solidity can return the entire array.
20      // But this function should be avoided for
21      // arcays thst can grow indefinitely in length.
22      function getArr() public view returns (uint[3] memory) {    infinite
23          return arr3;
24      }
25
```

Explain contract

0 | Listen on all transactions | Filter with trans

Type the library name to see available commands.

`

- Tutorial no.13

LEARNETH

Tutorials list | Syllabus

8.2 Data Structures - Mappings
12 / 19

with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer | Show answer

Next

Well done! No errors.

Compile | $ inputsAndOutputs.sol | $ visibility.sol | $ ifElse.sol | $ arrays.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.25;
3  //Bhagyesh Patil D28A 03
4
5  contract Mapping {
6      // Mapping from address to uint
7      mapping(address => uint) public balances;
8
9      function get(address _addr) public view returns (uint) {    2872 gas
10         // Mapping always returns a value.
11         // If the value was never set, it will return the default value.
12         return balances[_addr];
13     }
14
15     function set(address _addr) public {    25256 gas
16         // Update the value at this address
17         balances[_addr] = _addr.balance;
18     }
19
20     function remove(address _addr) public {    5566 gas
21         // Reset the value to the default value.
22         delete balances[_addr];
```

Explain contract

0 | Listen on all transactions | Filter with trans

Type the library name to see available commands.

- Tutorial no.14

LEARNETH

Tutorials list | Syllabus

8.3 Data Structures - Structs
14 / 19

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter a deletes a struct member with the given index in the `todos` mapping.

Check Answer | Show answer

Next

Well done! No errors.

Compile | its.sol | visibility.sol | ifElse.sol | loops.sol | arrays.sol | mappingc.sc

```
35
36     // update text
37     function update(uint _index, string memory _text) public {    infinite gas
38         Todo storage todo = todos[_index];
39         todo.text = _text;
40     }
41
42     // update completed
43     function toggleCompleted(uint _index) public {    28995 gas
44         Todo storage todo = todos[_index];
45         todo.completed = !todo.completed;
46     }
47     //Bhagyesh Patil D28A 60
48     function toggleCompleted:(uint _index) public {    28995 gas
49         Todo storage todo = todos[_index];
50         todo.completed = !todo.completed;
31     }
52
53     // update remove
54     function remove(uint _index) public {    25666 gas
54         delete todos[_index];
55     }
```

Explain contract

0 | Listen on all transactions | Filter with transaction hash

Type the library name to see available commands.

`

- Tutorial no.15

LEARNETH

Tutorials list | Syllabus

8.4 Data Structures - Enums
15 / 19

and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | ility.sol | ifElse.sol | loops.sol | arrays.sol | mappings.sol

```
14      // Default value is the first element listed in
15      // definition of the type, in this case "Pending"
16      Status public status;
17
18      //Aryan Patankar D20A 38
19      enum Size {
20          S,
21          M,
22          L
23      }
24
25      Size public sizes;
26
27      function getSize() public view returns (Size) {    2633 gas
28          return sizes;
29      }
30
31
32      // Returns uint
33      // Pending  - 0
34      // Shipped  - 1
35      // Accepted - 2
```

Explain contract

0 | Listen on all transactions | Filter with transac

Type the library name to see available commands.

- Tutorial no.16

LEARNETH

Tutorials list | Syllabus

9. Data Locations
16 / 19

to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location *memory* inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location *memory* inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function f return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer | Show answer

Compile | ifElse.sol | loops.sol | arrays.sol | mappings.sol | structs.sol | enums.sol | dataLocations.s

```
12      //Aryan Patankar D20A 38
13      function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory) {    infinite gas
14          // call _f with state variables
15          _f(arr, map, myStructs[1]);
16
17          // 1. get a struct from a mapping (Storage reference)
18          MyStruct storage myStruct = myStructs[1];
19          // Change the value of the myStruct member foo to 4
20          myStruct.foo = 4;
21
22          // 2. create a struct in memory
23          MyStruct memory myMemStruct = MyStruct(0);
24
25          // Create myMemStruct2 (Memory reference)
26          MyStruct memory myMemStruct2 = myMemStruct;
27          // Change myMemStruct2 member foo to 1
28          myMemStruct2.foo = 1;
29
30          // 3. Create myMemStruct3 (Independent copy from storage)
31          MyStruct memory myMemStruct3 = myStruct;
32          myMemStruct3.foo = 3;
33
34          // 4. Return all three structs
35          return (myStruct, myMemStruct2, myMemStruct3);
36      }
```

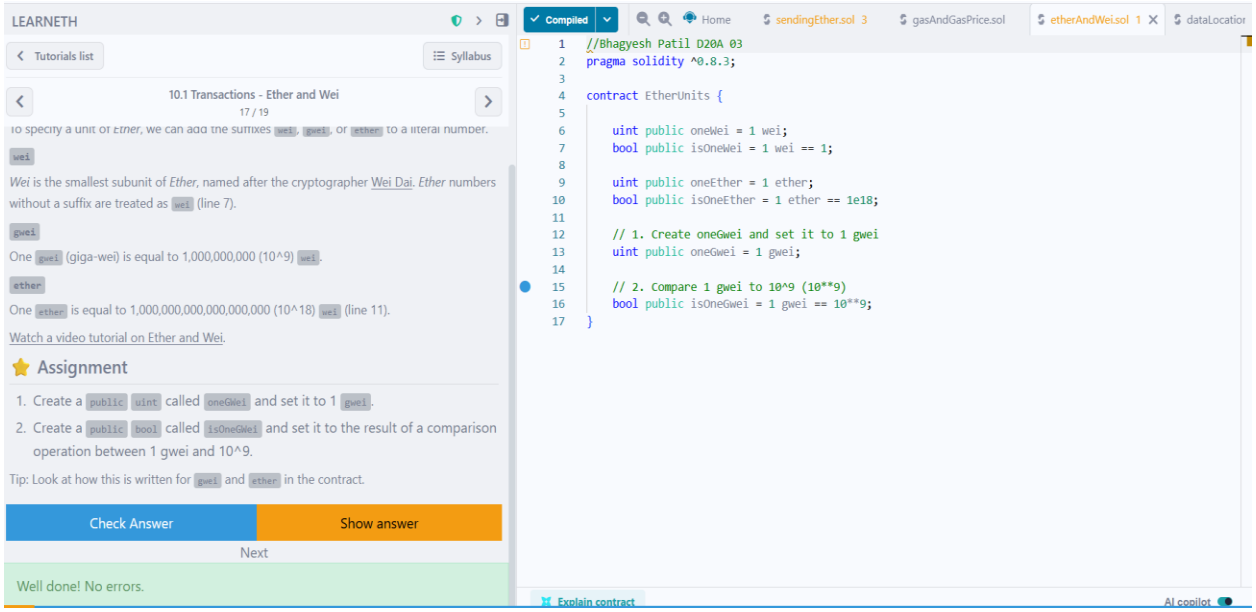
Explain contract

0 | Listen on all transactions | Filter with transaction hash or ad...

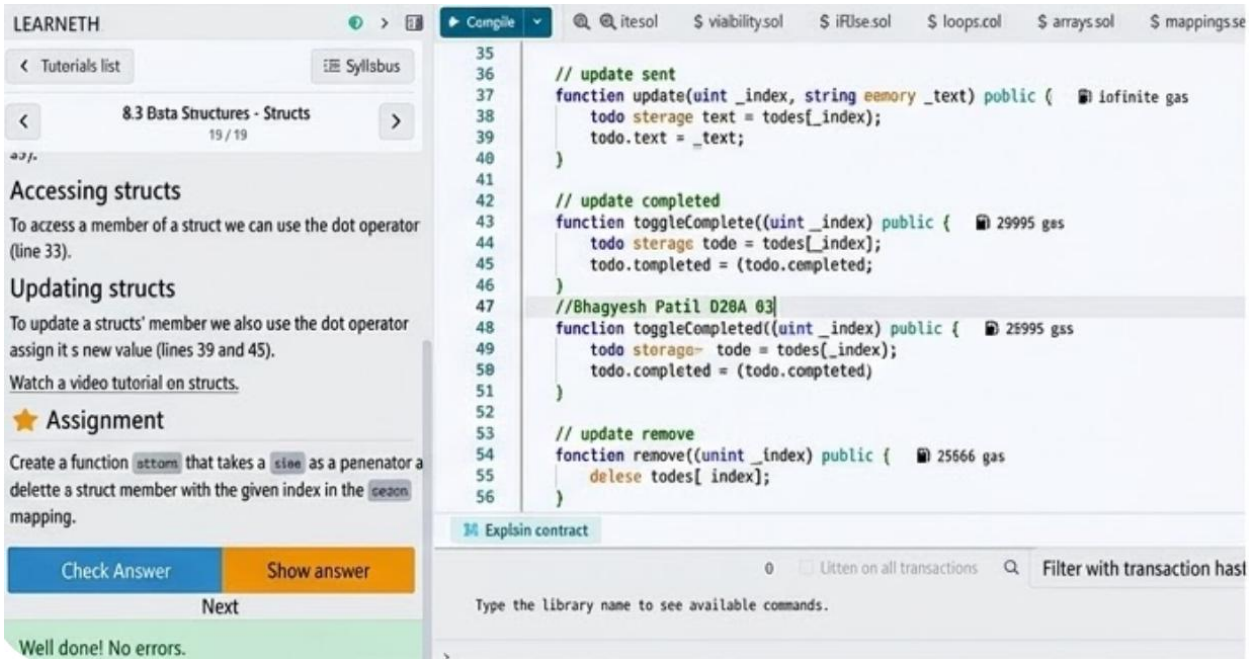
Type the library name to see available commands.

`

- Tutorial no.17

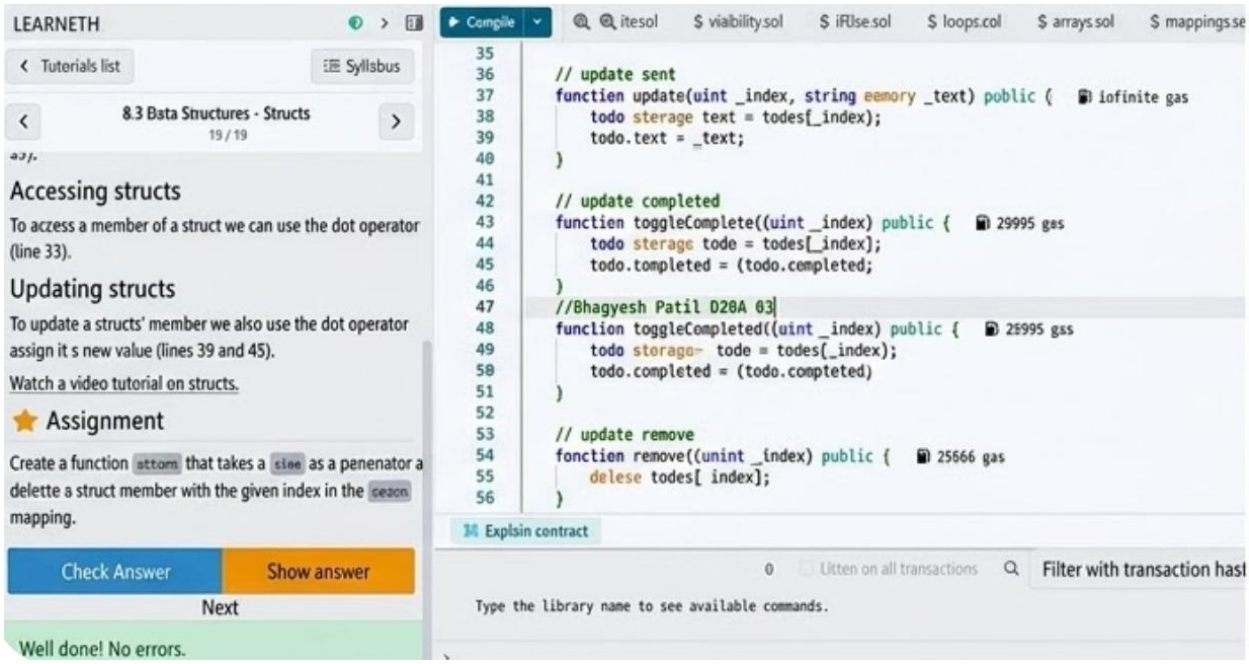


- Tutorial no.18



`

- Tutorial no.19



Conclusion:-

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.